VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report

Hospital Waiting-Time Analysis

A9513-Python Programming Laboratory

Submitted by

Batch No. 1

Abhinav. S
(25881A05BZ)
Akhil. G
(25881A05CC)
Akshara Reddy. T
(25881A05CD)

Course Facilitator

Mr. M. Raghavendra
Assistant Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering,

Hyderabad

November 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

Vibrant Vardhaman
25 Years of

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Hospital Waiting-Time Analysis"** is carried out by **"Abhinav.S" , "Akhil.G" and "Akshara Reddy.T" with** Roll Numbers **"25881A05BZ", "25881A05CC" and "25881A05CD"** towards **A9513 – Python Programming Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2026-27.

Signature of the Course Faculty
Mr. M.Ragavendra
Assistant Professor, CSE

Signature of the HOD
Dr. Y. Vijaylatha Reddy
Professor, HOD, CSE

Contents

1 Introduction 3
2 Literature Review 3
3 Objectives 3
4 Methodology 3
4.1 Architectural Diagram 3
4.2 Hardware and Software Requirements 3
4.3 Working Principle 4
4.4 Software Implementation 4
5 Results and Discussion 4
6 Mapping POs and SDGs 5
6.1 Program Outcomes (POs) Mapping 5
6.2 Sustainable Development Goals (SDGs) Mapping 5
7 Conclusion 5
Bibliography 6

Abstract

Hospital Waiting-Time Analysis is a Python-based project designed to analyze the time patients spend waiting for medical consultation in hospitals. Long waiting times can cause inconvenience to patients and affect the efficiency of hospital services. The main objective of this project is to calculate and analyze patient waiting times using data such as patient arrival time, consultation start time, department, and service duration. Python libraries such as Pandas and Matplotlib can be used to process the data, perform calculations, and represent the results through graphs and charts.

This project helps identify peak hours, departments with longer waiting times, and patterns that contribute to delays in hospital services. By analyzing these factors, the project aims to provide useful insights that can support better resource allocation, improve patient flow, and enhance hospital management. The results can help hospitals understand waiting-time patterns and explore ways to reduce delays, leading to more efficient healthcare services and improved patient satisfaction.

Additionally, the project provides a data-driven approach to understanding hospital operations and improving the overall patient experience. The analyzed results can be used to compare waiting times across different departments and time periods, helping identify areas that require improvement. The project also demonstrates how Python programming can be applied to real-world healthcare problems through data processing, statistical analysis, and graphical representation. Although the accuracy of the findings depends on the quality of the available data, the analysis can serve as a useful starting point for developing strategies to manage patient queues, utilize hospital resources effectively, and provide timely medical services.

Keywords: Hospital Waiting Time, Python, Data Analysis, Patient Flow, Pandas, Matplotlib, Healthcare Management, Waiting-Time Prediction, Data Visualization, Hospital Efficiency.

1 Introduction

Hospital Waiting-Time Analysis is a Python-based project that focuses on studying the time patients spend waiting to receive medical consultation or treatment in a hospital. Waiting time is an important factor in healthcare management because long delays can affect patient satisfaction and the efficient use of hospital resources. The main concept of this project is to collect, organize, and analyze patient-related data, including arrival time, consultation start time, department, and service duration. Python programming is used to process the data and identify patterns in patient waiting times. Libraries such as Pandas help in data handling and calculation, while Matplotlib is used to present the results through graphs and charts. This project demonstrates how data analysis can be applied to solve practical problems in the healthcare sector.

The project involves mathematical concepts such as arithmetic operations, averages, percentages, and basic statistics. Patient waiting time can be calculated by subtracting the arrival time from the consultation start time. The average waiting time is calculated by dividing the total waiting time of all patients by the total number of patients. For example, if five patients have a combined waiting time of 150 minutes, the average waiting time is 30 minutes. Statistical measures such as minimum, maximum, and average waiting times help compare hospital departments and identify periods of high patient demand. The project also uses logical conditions, data grouping, and graphical representation to interpret the results. By combining Python programming, mathematical calculations, and statistical analysis, the project aims to identify waiting-time patterns and provide useful information for improving hospital efficiency and patient flow.

2 Literature Review

Hospital Waiting-Time Analysis is an important area of study in healthcare management, as waiting time directly affects patient satisfaction and the efficiency of hospital services. Existing studies on hospital management focus on reducing patient waiting times through better appointment scheduling, efficient staff allocation, queue management, and improved patient flow. Traditional methods of monitoring waiting times often involve manual records and basic calculations, which can be time-consuming and may not provide a clear understanding of waiting-time patterns. Recent approaches use data analysis techniques to study patient arrival patterns, consultation duration, department-wise delays, and peak hours to identify areas that require improvement.

The proposed project uses Python programming and data analysis techniques to examine hospital waiting times systematically. Python libraries such as Pandas can be used to organize and process patient data, while Matplotlib can represent the results using graphs and charts. Mathematical and statistical methods, including average, minimum, maximum, and comparative analysis, help identify waiting-time trends and differences between departments. Compared with manual analysis, this approach makes it easier to process larger datasets, visualize patterns, and interpret results. The project aims to provide useful insights that can support better queue

management, resource utilization, and decision-making in hospitals. Its findings depend on the availability and accuracy of the data used for analysis.

3 Objectives

- To calculate and analyze patient waiting times using Python based on arrival time and consultation start time.
- To identify peak hours, department-wise delays, and patterns in hospital waiting times using statistical analysis and data visualization.
- To provide useful insights that help improve patient flow, hospital resource utilization, and the efficiency of healthcare services.

4 Methodology

4.1 Architectural Diagram

The architecture of the Hospital Waiting-Time Analysis project illustrates the flow of data from patient information input to the final analysis and visualization. The system uses Python to process patient data, calculate waiting times, and identify patterns such as average waiting time, department-wise delays, and peak hours.

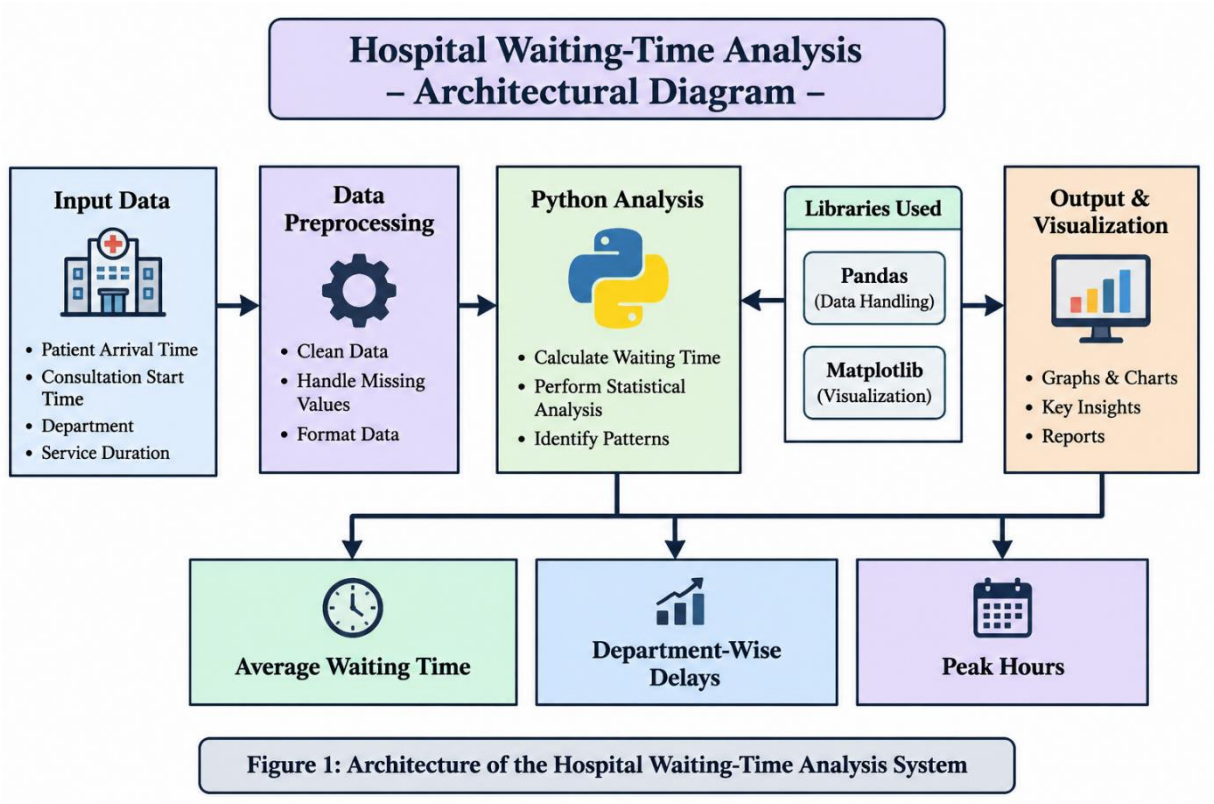


Figure 1: Architecture of the Hospital Waiting-Time Analysis System

4.2 Hardware and Software Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent.
- RAM: Minimum 4 GB.
- Storage: At least 500 MB of free disk space.
- Operating System: Windows 10/11 or equivalent.
- Input Devices: Keyboard and mouse.
- Output Device: Monitor or display.

Software Requirements

- Programming Language: Python 3.x.
- IDE / Code Editor: Visual Studio Code or IDLE.
- Data Analysis Library: Pandas.
- Numerical Computation Library: NumPy.
- Data Visualization Library: Matplotlib.
- Data Storage: CSV or Excel files.
- Operating System: Windows, Linux, or macOS.

4.3 Working Principle

The Hospital Waiting-Time Analysis project uses Python programming to process patient data and calculate waiting times. Variables and data types are used to store patient details, arrival times, consultation start times, and waiting times. Conditional statements and loops help process multiple patient records and identify patients who experience longer waiting periods. Functions are used to organize the program into reusable sections, making the code easier to understand, maintain, and execute.

Python libraries such as Pandas, NumPy, and Matplotlib are used to perform data analysis and visualization. Pandas helps read and organize patient data from CSV files, while NumPy supports numerical calculations. Matplotlib is used to create graphs and charts that display average waiting times, department-wise delays, and peak hours. These programming features automate the analysis process, reduce manual calculations, and help present meaningful results that can support better hospital management and improve patient flow.

Additionally, Python provides flexibility and scalability, allowing the project to handle patient records of different sizes. File-handling features enable the program to import data from external files and save the analyzed results for future reference. Error-handling techniques can also be implemented to manage missing or invalid data and prevent unexpected program interruptions. By combining these features, the project provides a simple and effective way to analyze hospital waiting times, understand patient flow patterns, and identify areas where hospital services can be improved.

4.4 Software Implementation

```
import pandas as pd
import matplotlib.pyplot as plt

# Sample hospital patient data
data = {
    "Patient": ["P1", "P2", "P3", "P4", "P5"],
    "Department": [
        "General", "Cardiology", "General",
        "Orthopedics", "Cardiology"
    ],
    "Arrival_Time": [
        "09:00", "09:10", "09:15",
        "09:20", "09:30"
    ],
    "Consultation_Start": [
        "09:20", "09:40", "09:35",
        "09:50", "10:00"
    ]
}

df = pd.DataFrame(data)

# Convert time columns into datetime format
df["Arrival_Time"] = pd.to_datetime(
    df["Arrival_Time"], format="%H:%M"
)
df["Consultation_Start"] = pd.to_datetime(
    df["Consultation_Start"], format="%H:%M"
)

# Calculate waiting time in minutes
df["Waiting_Time"] = (
    df["Consultation_Start"] - df["Arrival_Time"]
).dt.total_seconds() / 60

# Display patient waiting times
print("Patient Waiting-Time Analysis")
print(df[["Patient", "Department", "Waiting_Time"]])

# Calculate overall statistics
print("\nAverage Waiting Time:",
      df["Waiting_Time"].mean(), "minutes")
print("Minimum Waiting Time:",
      df["Waiting_Time"].min(), "minutes")
print("Maximum Waiting Time:",
      df["Waiting_Time"].max(), "minutes")

# Calculate department-wise average waiting time
department_avg = df.groupby(
    "Department"
```

```
)["Waiting_Time"].mean()

print("\nDepartment-wise Average Waiting Time:")
print(department_avg)

# Plot department-wise waiting times
department_avg.plot(kind="bar", color="skyblue")
plt.title("Average Waiting Time by Department")
plt.xlabel("Department")
plt.ylabel("Waiting Time (minutes)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

The program begins by importing Pandas for data processing and Matplotlib for visualization. It creates a sample dataset containing patient IDs, department names, arrival times, and consultation start times. The time values are converted into a suitable datetime format so that the difference between arrival and consultation start times can be calculated accurately in minutes.

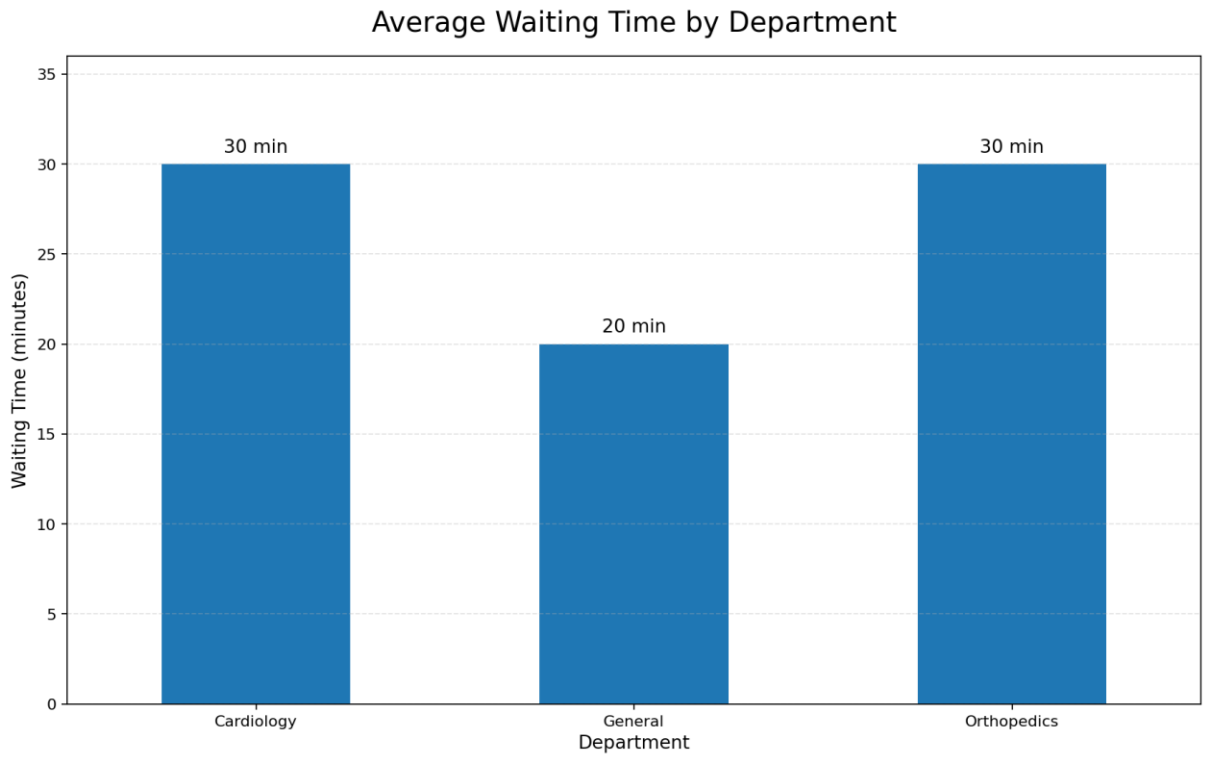
Next, the program calculates the average, minimum, and maximum waiting times for all patients. It also groups the records by department to determine the average waiting time in each department. Finally, Matplotlib generates a bar chart to compare the departments. This helps identify departments with longer waiting times and provides a simple way to understand delays in hospital services.

The program also uses the Pandas library to group patient records according to their departments and calculate the average waiting time for each department. It displays the overall average, minimum, and maximum waiting times to help understand patient delays. The Matplotlib library is used to create a bar graph that compares waiting times across departments. These results help identify departments with longer waiting periods and provide useful information for improving patient flow and hospital management.

5 Results and Discussion

```
Patient Waiting-Time Analysis
  Patient   Department  Waiting_Time
0      P1      General          20.0
1      P2   Cardiology          30.0
2      P3      General          20.0
3      P4  Orthopedics          30.0
4      P5   Cardiology          30.0
```

```
Department
Cardiology     30.0
General        20.0
Orthopedics    30.0
Name: Waiting_Time, dtype: float64
```



The program was tested using five sample patient records from three hospital departments. The overall average waiting time was 26 minutes, with a minimum waiting time of 20 minutes and a maximum of 30 minutes. Cardiology and Orthopedics had an average waiting time of 30 minutes, while General had an average waiting time of 20 minutes.

Interpretation

The results show that waiting times vary between hospital departments. Cardiology and Orthopedics have higher average waiting times than General,

indicating that these departments may require further investigation to understand the reasons for delays. The bar chart makes these differences easier to identify and compare**.**

Achievement

The objectives of the project were achieved by calculating patient waiting times, determining overall statistical measures, and comparing average waiting times across departments. The project also demonstrates how Python libraries such as Pandas and Matplotlib can be used to analyze patient data and present results visually.

Implications

The findings can help hospital administrators identify departments that may need improved scheduling, better staff allocation, or more effective queue management. By analyzing larger datasets collected over different days and times, hospitals could identify peak periods and plan resources more effectively. However, the current results are based on a small sample dataset and should not be treated as evidence about actual hospital performance.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied computational thinking and data-handling principles (such as datetime parsing and arithmetic) to model and analyze hospital operational logistics.
PO2	Problem Analysis	Formulated and analyzed raw timestamp logs to extract key performance metrics—such as minimum, maximum, and average waiting times—across different departments.
PO3	Design/Development of Solutions	Designed a structured data pipeline using Python to clean, transform, and evaluate patient flow, identifying department-specific operational bottlenecks (e.g., comparing Cardiology and General wait times)
PO4	Conduct Investigations of Complex Problems	Investigated patient flow inefficiencies and variable wait times across different hospital departments to diagnose scheduling bottlenecks using data-driven inquiry.
PO5	Engineering Tool Usage	Utilized industry-standard programming environments and libraries (Python, Pandas, and Matplotlib) to process tabular datasets and generate visual data charts.

PO6	The Engineer and The World	Addressed real-world healthcare challenges such as hospital congestion and patient wait-times, demonstrating how data solutions impact public health and societal well-being.
PO7	Ethics	Ensured ethical handling of simulated patient records, maintaining data integrity and objective, unbiased reporting of operational performance without misrepresenting statistics.
PO8	Individual and Collaborative Team work	Demonstrated the ability to independently design and execute a data-driven system, or collaborate effectively within a multidisciplinary healthcare operations and IT team.
PO9	Communication	Effectively communicated analytical findings and department-wise performance metrics through formatted console outputs and clear graphical bar charts.
PO10	Project Management and Finance	Linked operational efficiency to hospital administration and financial management, as reducing patient wait times directly optimizes resource utilization and overhead costs.
PO11	Life-Long Learning	Exhibited a commitment to continuous learning by adopting modern data analytics tools (Python, Pandas, Matplotlib) and applying them to emerging domains like healthcare informatics.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 3	Good Health and Well-being	Directly contributes to improving healthcare quality and patient satisfaction by identifying bottlenecks and reducing overall waiting times for medical consultations.
SDG 8	Decent Work and Economic Growth	Enhances operational efficiency and workflow productivity within healthcare facilities, helping medical staff manage patient volume effectively without burnout.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation and data-driven innovation in healthcare infrastructure through automated data processing and analytical visualization tools.
SDG 10	Reduced Inequalities	Helps ensure fair, systematic, and transparent queue management across various medical departments (such as Cardiology, General, and Orthopedics), providing equitable access to care.

7 Conclusion

This project successfully demonstrates the application of Python-based data analytics to evaluate hospital operational efficiency by systematically tracking patient waiting times. By converting raw arrival and consultation logs into actionable metrics using Pandas and visual charts via Matplotlib, the system provides clear visibility into operational bottlenecks across different hospital departments.

The implementation of this project centers on developing a Python-based data analytics pipeline to track, evaluate, and visualize hospital operational efficiency. By leveraging the **Pandas** library, raw patient timestamp logs were converted into standard datetime objects to automatically calculate individual waiting times ($WaitTime = ConsultationStart - ArrivalTime$). The system aggregates these metrics to output key performance statistics (minimum, maximum, and average wait times) both overall and across specific medical units (Cardiology, General, and Orthopedics). Finally, using **Matplotlib**, the processed metrics were synthesized into a clear bar chart, giving healthcare administrators an immediate, visual insight into departmental bottlenecks.

Bibliography

1. J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55.

2. W. McKinney, "Data Structures for Statistical Computing in Python," in *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61.

3. F. Nelli, *Python Data Analytics: With Pandas, NumPy, and Matplotlib*, 2nd ed. New York, NY, USA: Apress, 2018.

4. World Health Organization, *Monitoring the Building Blocks of Health Systems: A Handbook of Indicators and Their Measurement Strategies*, Geneva, Switzerland: World Health Organization, 2010.

5. United Nations, *Transforming Our World: The 2030 Agenda for Sustainable Development*, Resolution A/RES/70/1 adopted by the General Assembly, Sep. 25, 2015.